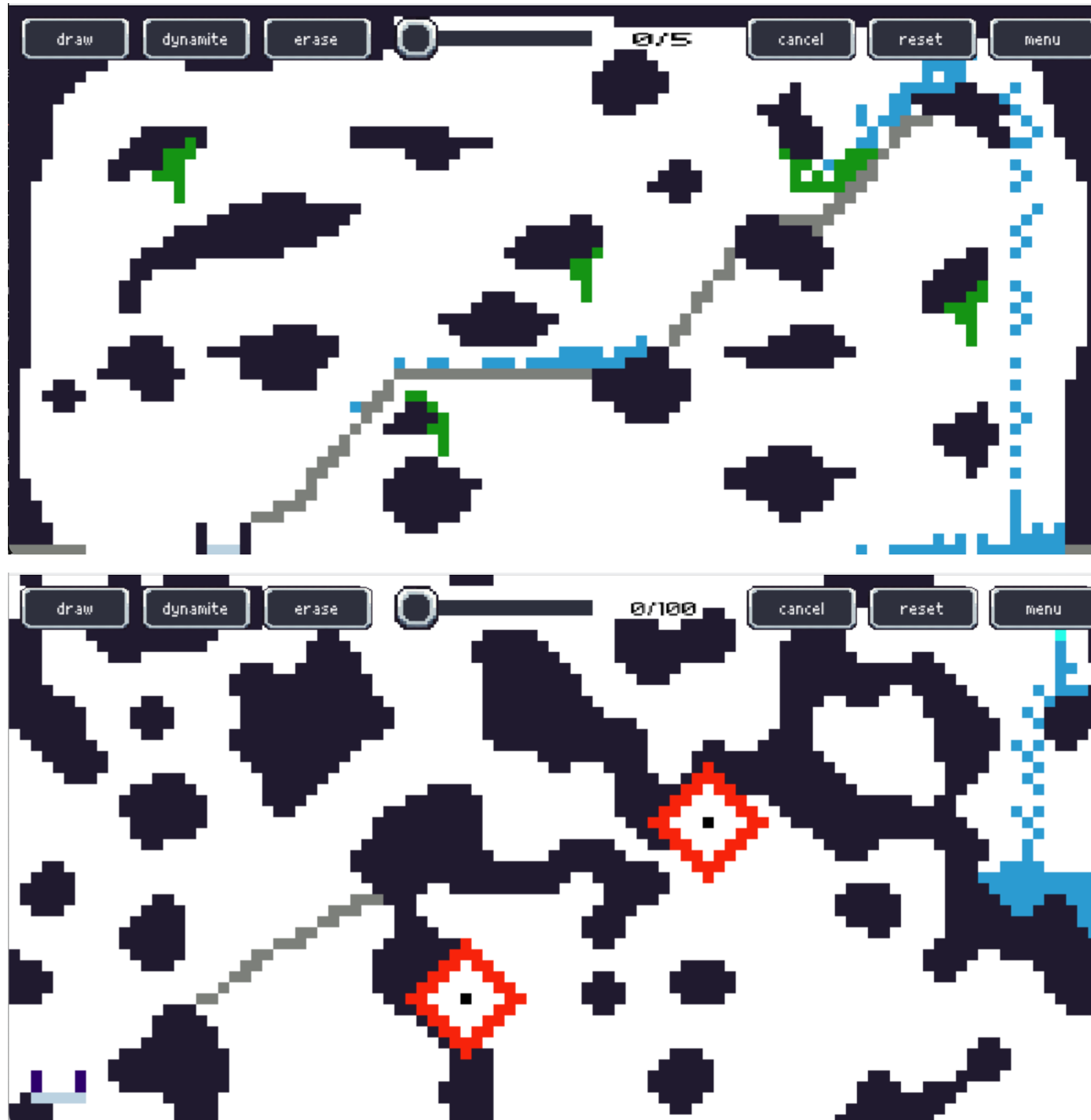


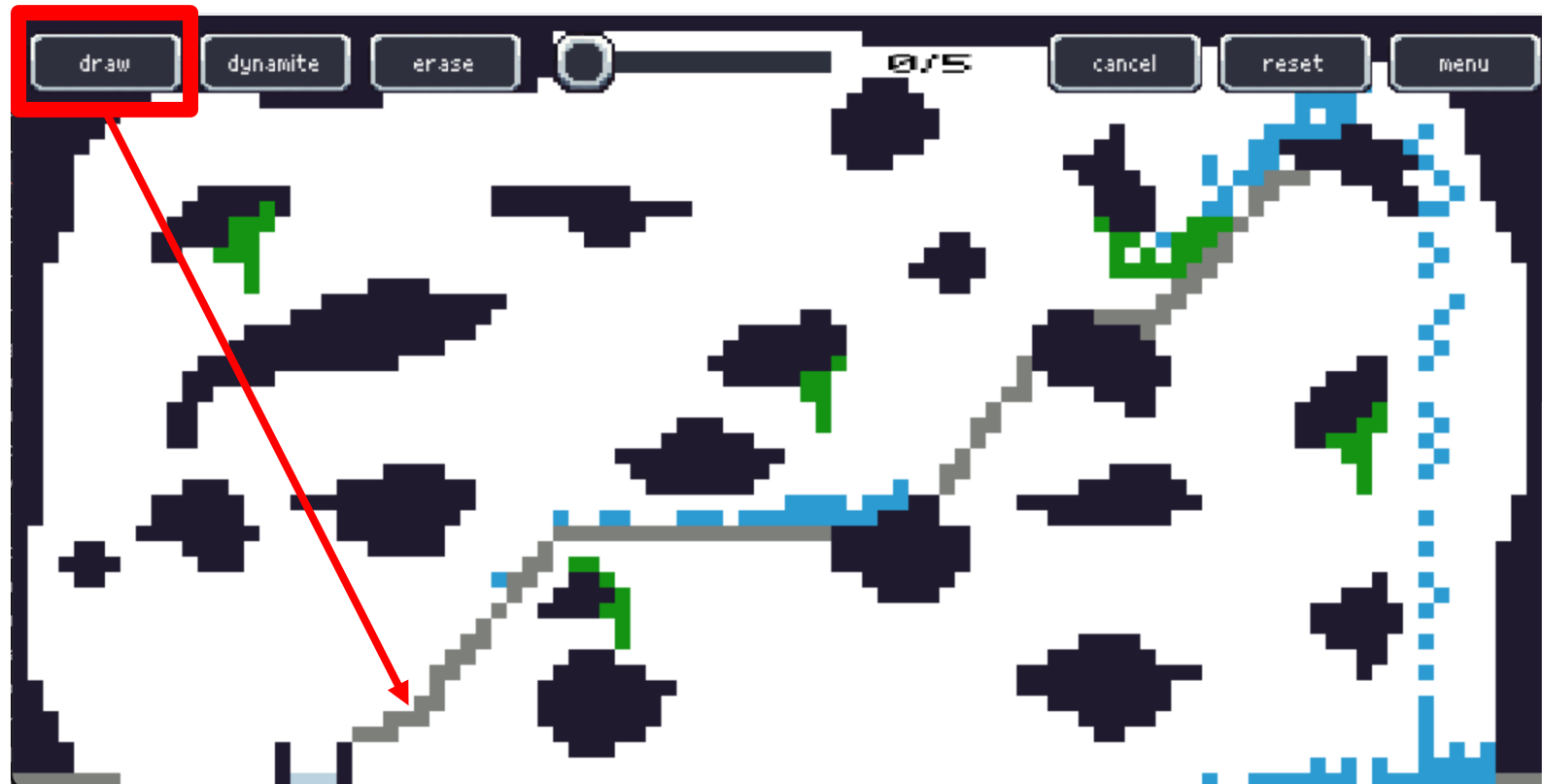
Jeu de la chute d'eau: là-haut

Groupe 22: Paul Vilars, Emile L'Excellent, Paul Verhaeghe, Maxime Joffroy, Tiziano Giannelli, Enguerrand de Jaegere

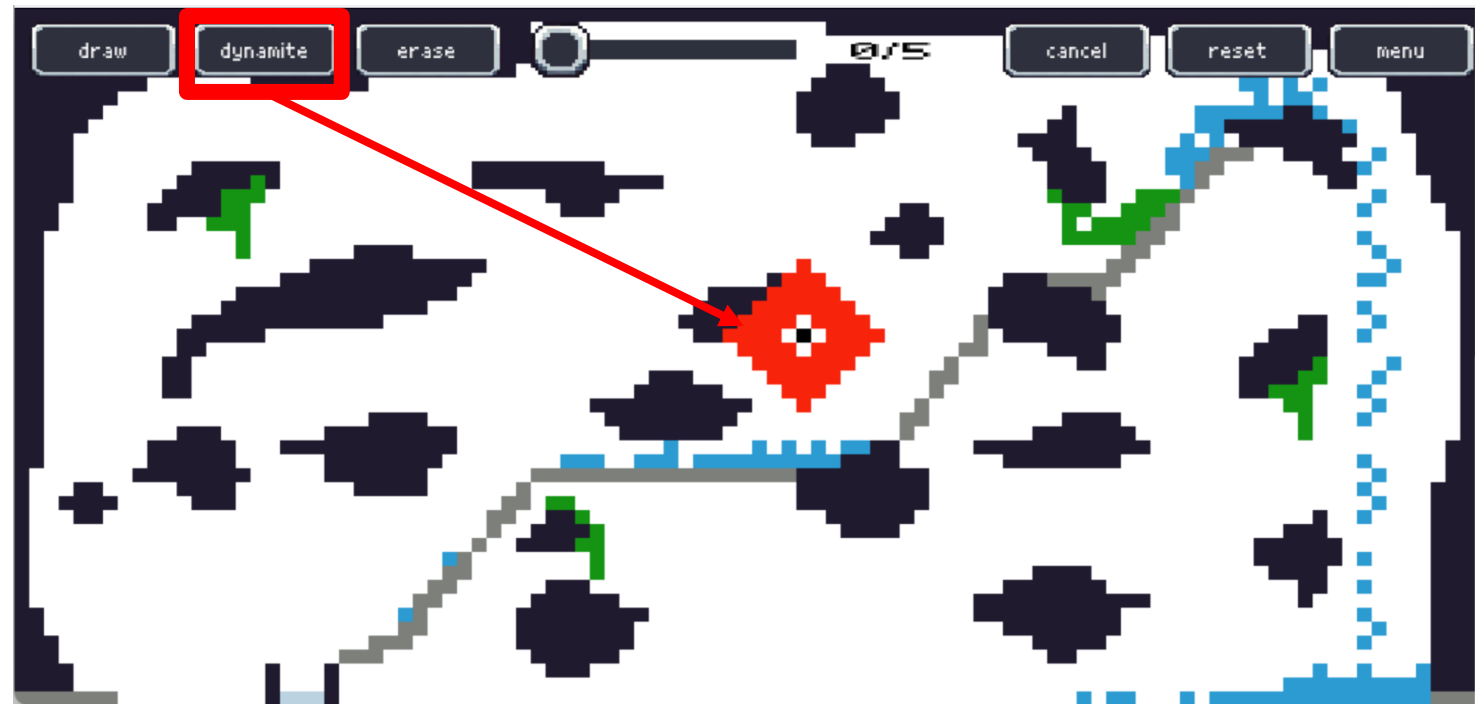


I/But et fonctionnement du jeu

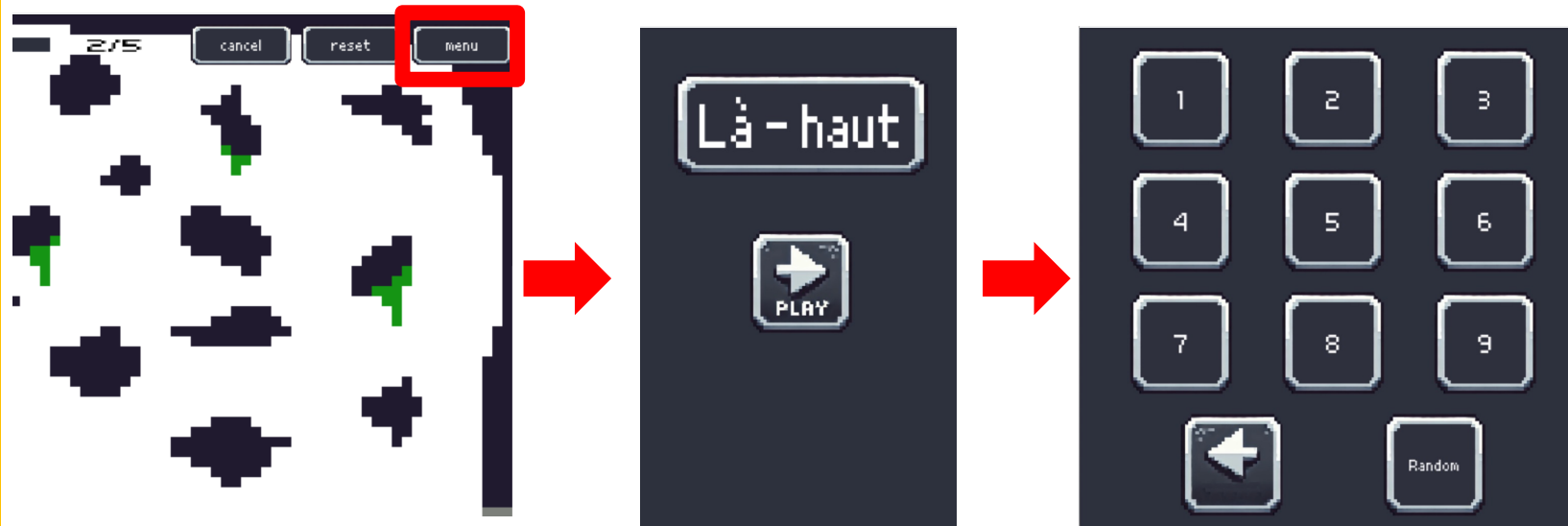
- **Objectif:** mettre un certain nombre de gouttes d'eau dans une certaine zone.
- **Lancement du jeu:** lance le programme `display_launcher.py`, puis pour gagner, l'utilisateur peut placer des blocs ou en effacer.
- **Présentation des différentes options du jeu:**



I/But et fonctionnement du jeu



- **Erase:** efface des blocs gris clair en cliquant dessus
- **Cancel:** retour au dernier arrêt

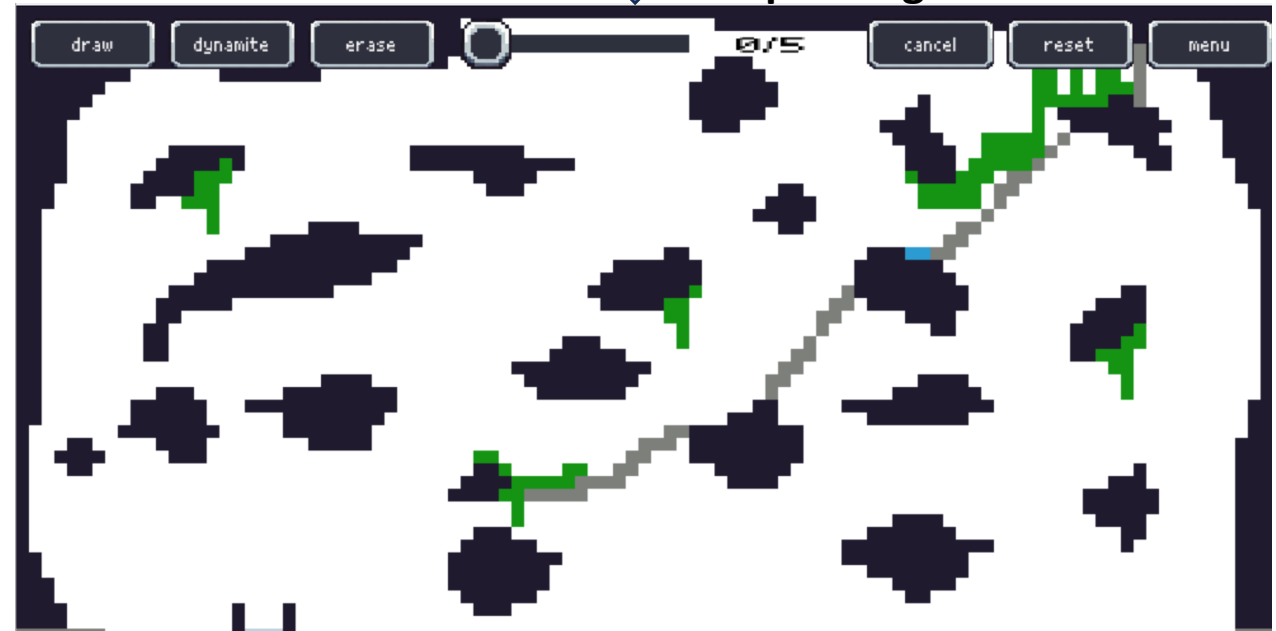


I/But et fonctionnement du jeu

- Types d'obstacles dans le jeu: **roche**/ **roche de l'utilisateur**/ **plante**

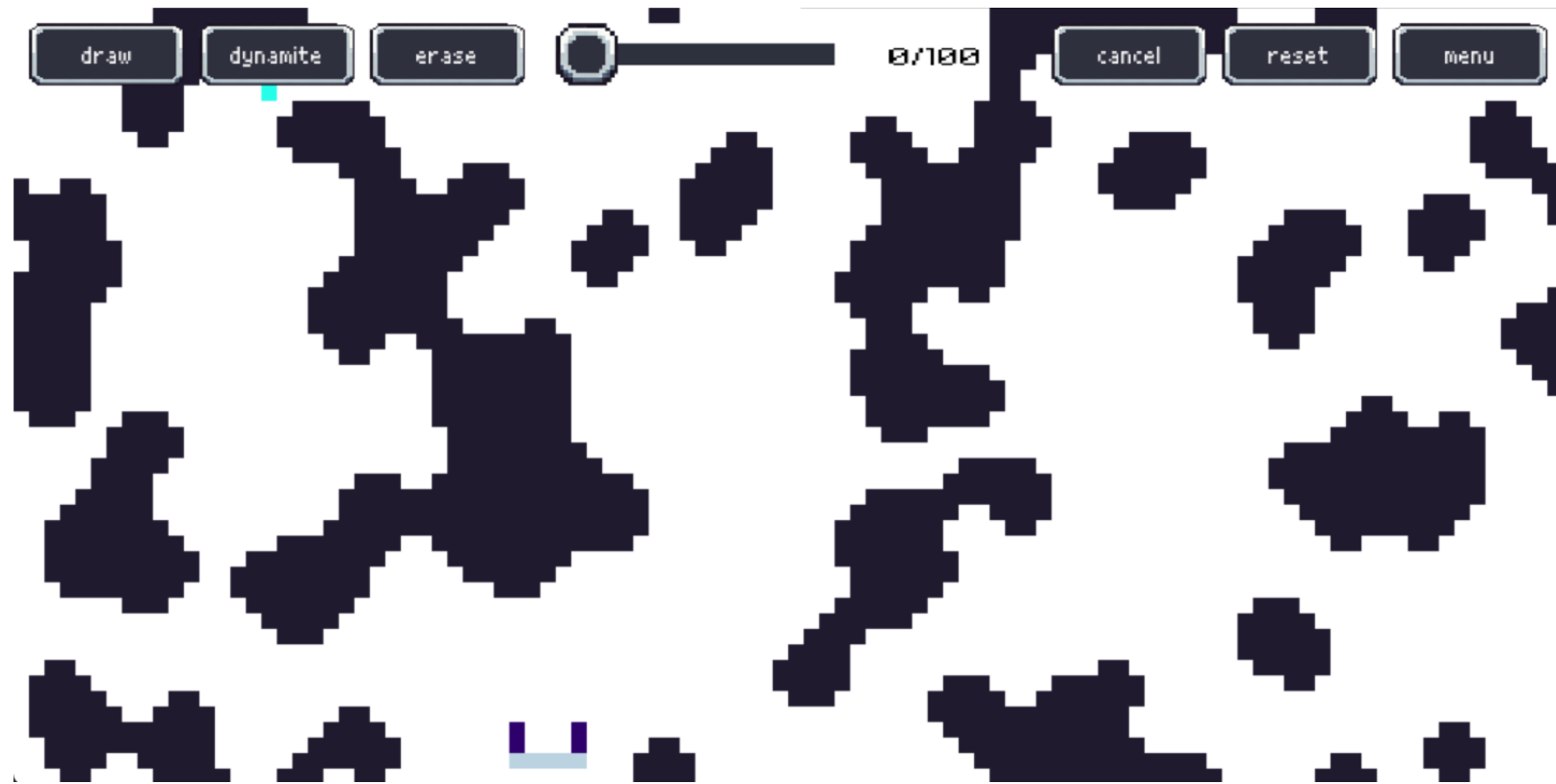


↓ Une plante grandit en absorbant de l'eau.



I/But et
fonctionnement
du jeu

Niveaux aléatoires : jeu de la majorité

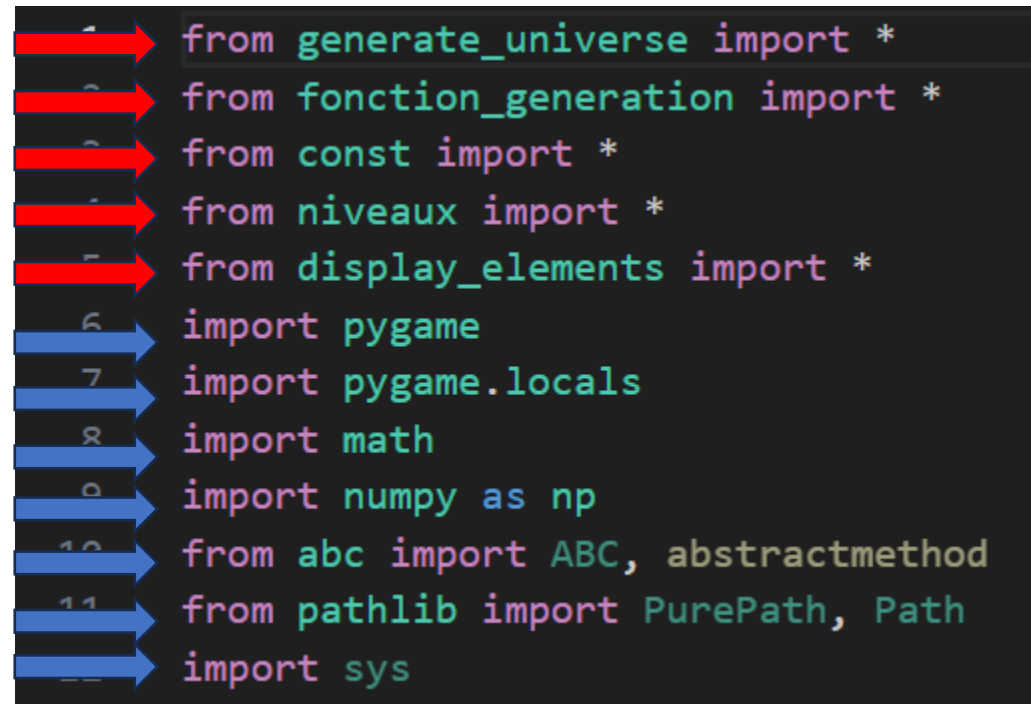


II/Analyse du code

- **Programme principal** : [display_launcher.py](#) , qui lance l'interface de jeu
- **Programme de création de niveaux** : [level_editor.py](#)
- **Une fonction génération** : met à jour l'univers
- 15 autres programmes appelés directement et indirectement par ces 2 programmes principaux
- Exemple, dans [display_in_game.py](#),

— Modules importés

— Programmes importés

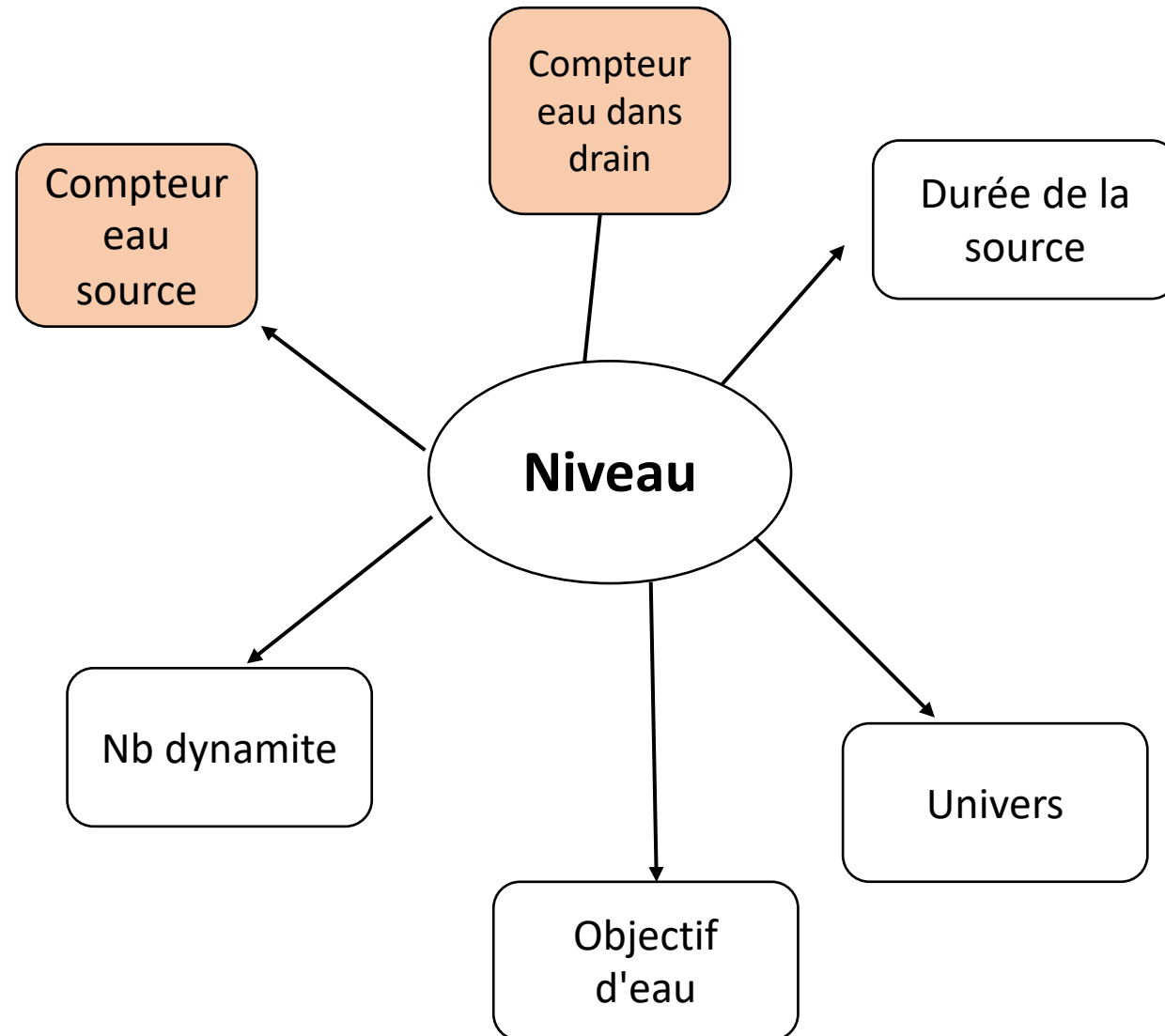


```
1 from generate_universe import *
2 from fonction_generation import *
3 from const import *
4 from niveaux import *
5 from display_elements import *
6 import pygame
7 import pygame.locals
8 import math
9 import numpy as np
10 from abc import ABC, abstractmethod
11 from pathlib import PurePath, Path
12 import sys
```

The image shows a snippet of Python code from the file `display_in_game.py`. The code is displayed on a dark background with syntax highlighting. Red arrows point to lines 1 through 5, which are 'from' imports from modules: `generate_universe`, `fonction_generation`, `const`, `niveaux`, and `display_elements`. Blue arrows point to lines 6 through 12, which are standard 'import' statements for `pygame`, `pygame.locals`, `math`, `numpy` (aliased as `np`), `abc` (with `ABC` and `abstractmethod`), `pathlib` (with `PurePath` and `Path`), and `sys`.

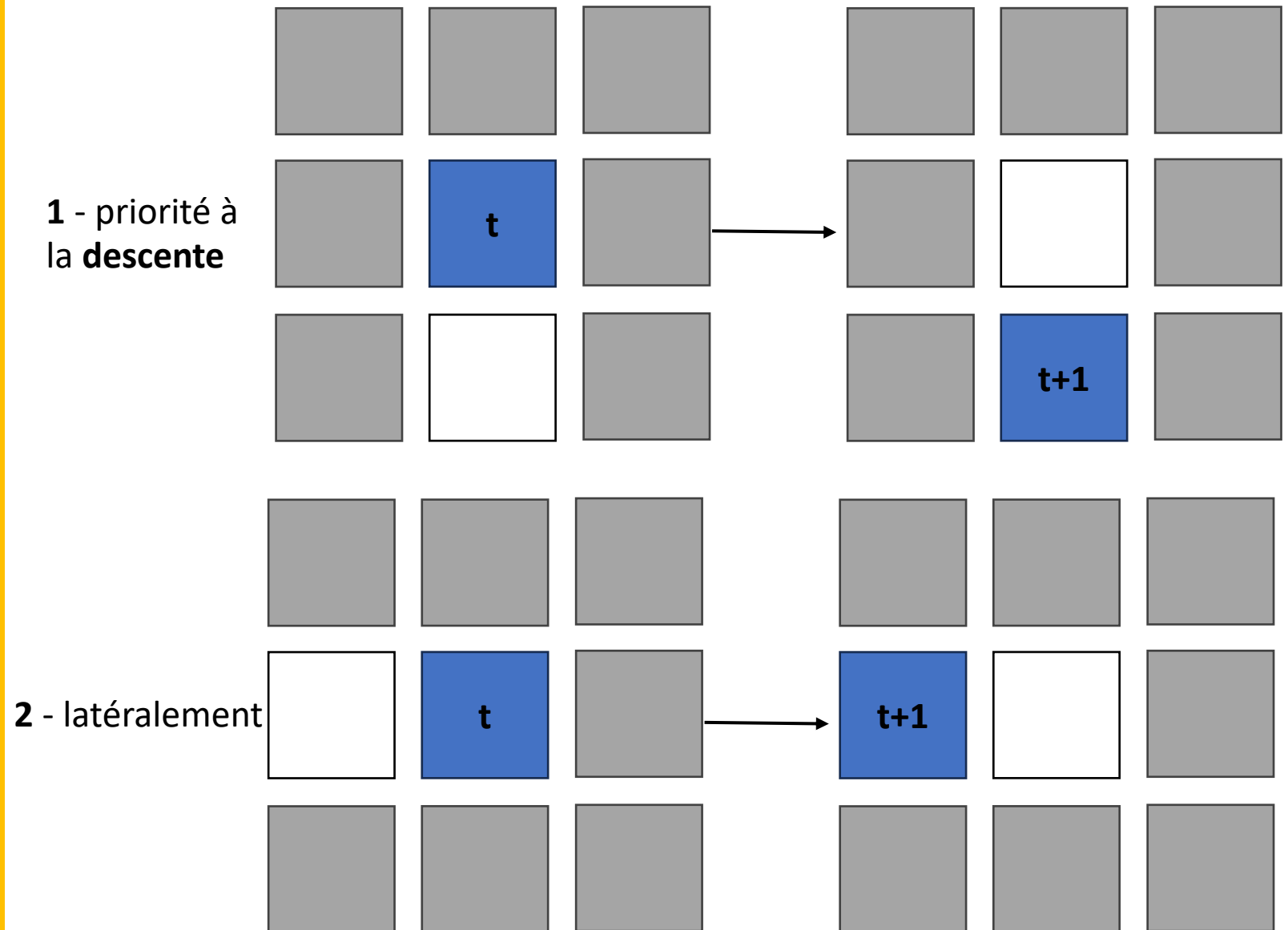
II/Analyse du code

Utilisation de la programmation orientée objet pour la gestion des niveaux



II/Analyse du code

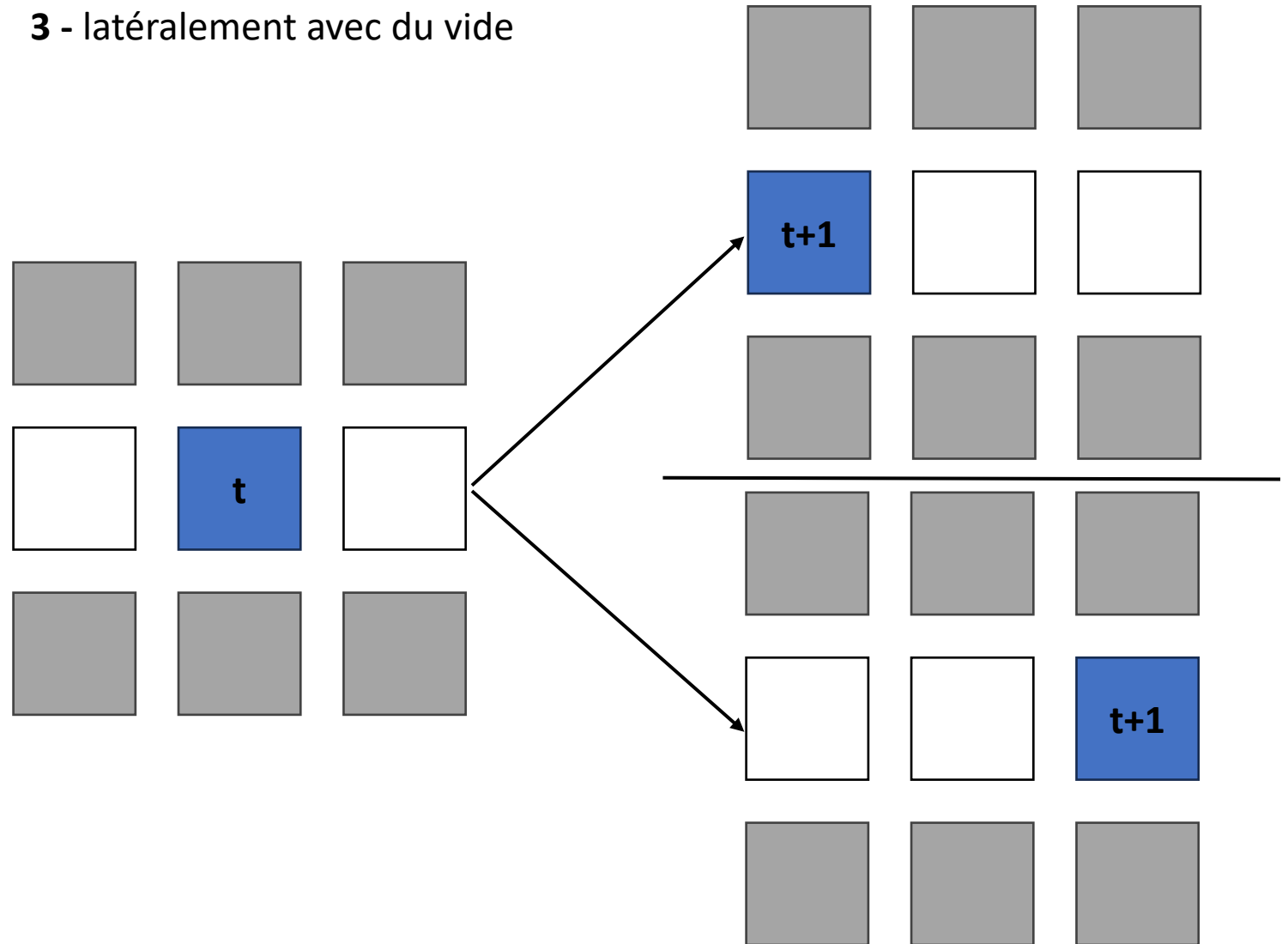
- **Mouvement de l'eau** : régis par un *automate cellulaire* suivant les règles suivantes



II/Analyse du code

- **Mouvement de l'eau** : régis par un *automate cellulaire* suivant les règles suivantes

3 - latéralement avec du vide

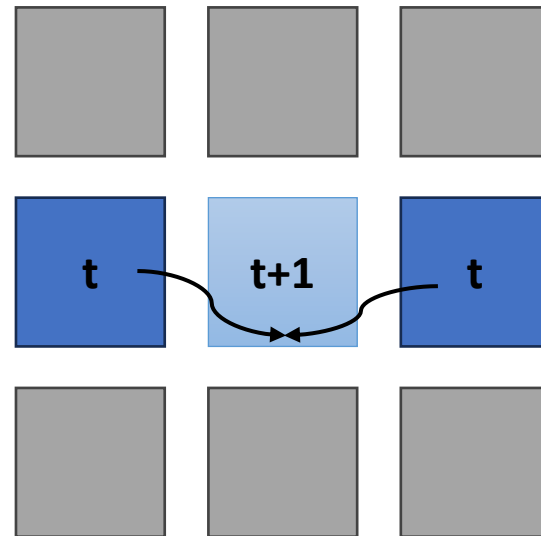


II/Analyse du code

- **Mouvement de l'eau** : régis par un *automate cellulaire* suivant les règles suivantes :
Solution : on prend aléatoirement celles qui partent à droite parmi celles qui ne descendent pas directement

Ex :

Problème (encore) : chevauchement



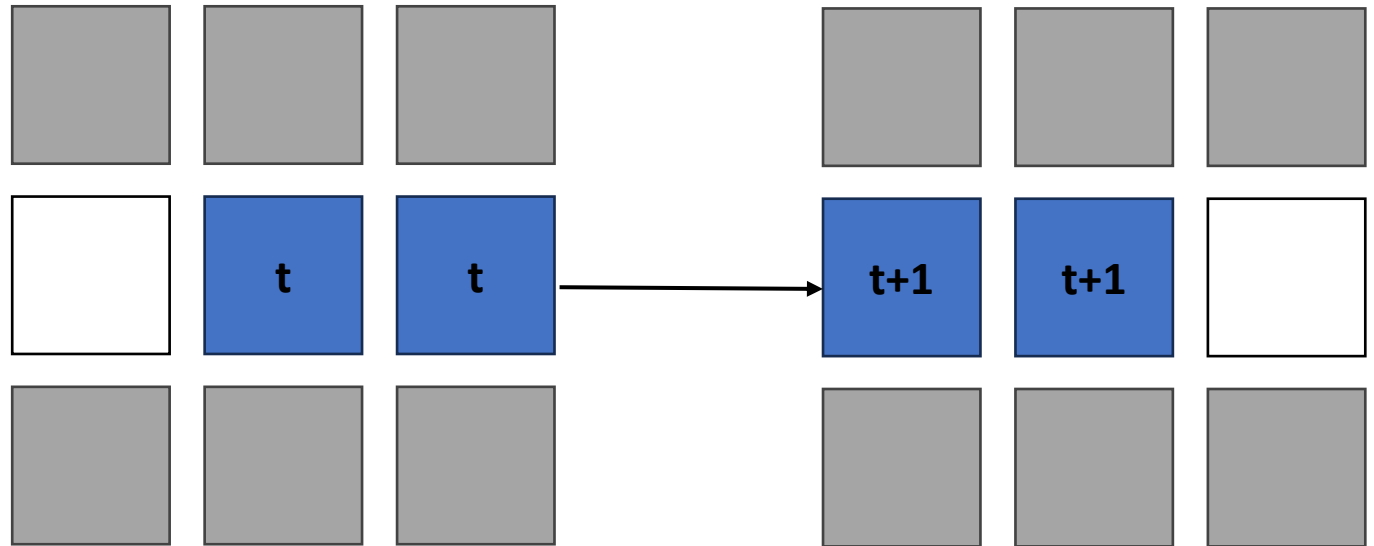
Solution : ne pas modifier tout de suite et garder en mémoire ceux qui vont à gauche et à droite

II/Analyse du code

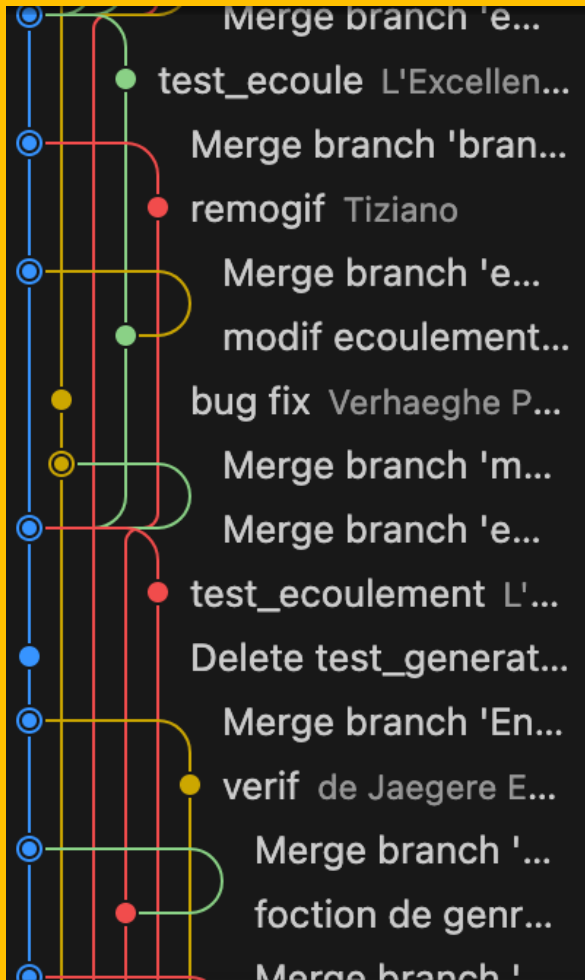
- **Mouvement de l'eau** : régis par un *automate cellulaire* suivant les règles suivantes :

Problème : l'eau part à gauche

Ex :

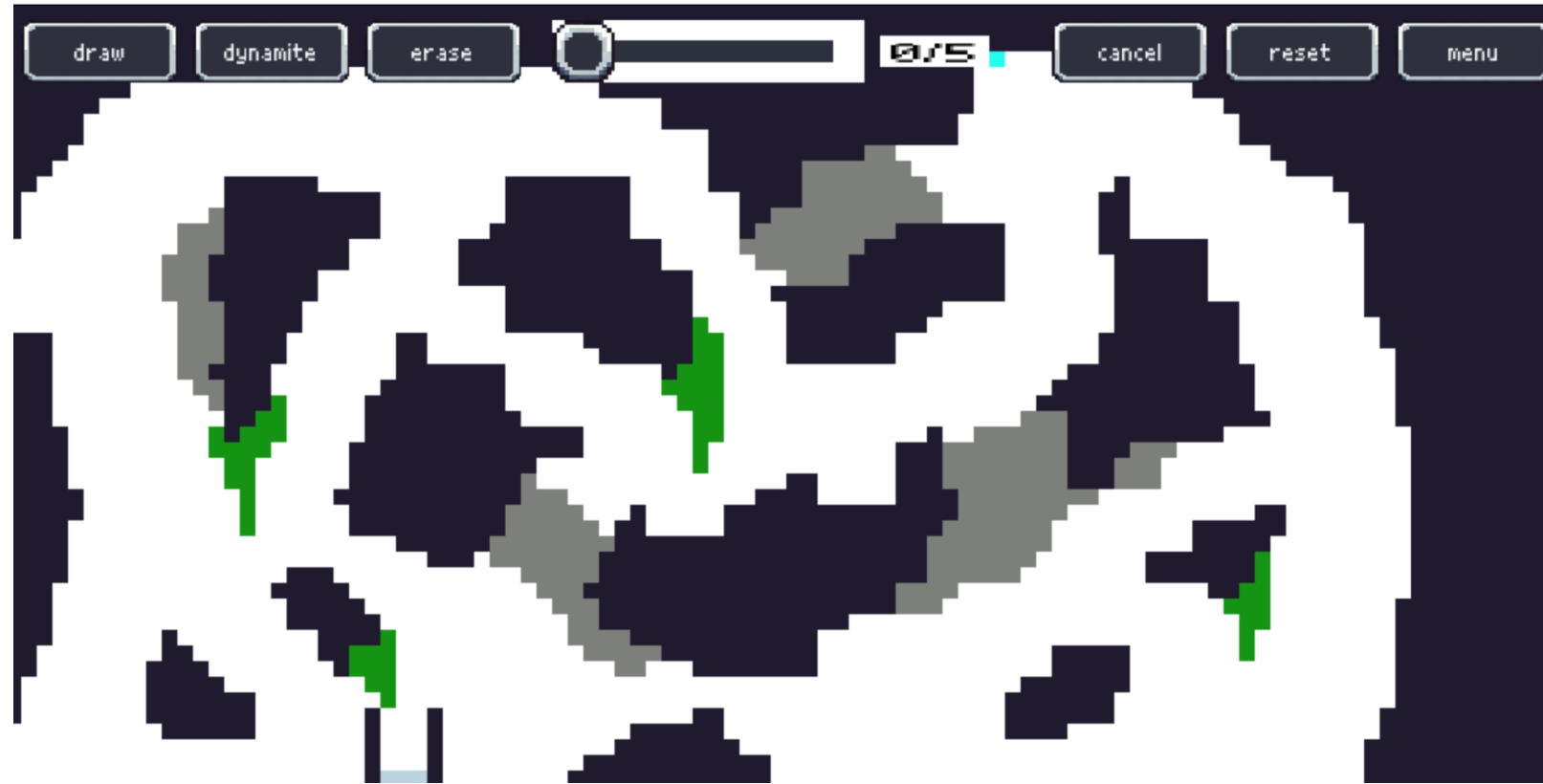


III/Mise en place: répartition du travail



- Répartition des tâches selon les compétences de chacun (aisance avec pygame, programmation objet, créativité des niveaux de jeu...).
- **Ordre de réalisation:** README et réalisation du MVP (fonctions de base telles que celle du mouvement de l'eau) -> ajout de nouvelles options (dynamite, plantes,...) et amélioration de l'interface graphique.
- Relecture des codes à plusieurs afin de les **tester** (création de fichiers tests) et de vérifier leur bon fonctionnement.
- **Mise en commun du travail:** Chacun travaillait sur sa branche puis après avoir sauvegardé (`git add fichier.py`, `git commit -m « fichier »`) et poussé les modifications (`git push`) on mergeait avec la branch master. Puis mise à jour de sa branche en rebasant master.
- Résolution des éventuels conflits avec VS code.

IV/Démonstration du produit



Conclusion: pistes d'amélioration

- Retravailler le fichier `level_editor.py` pour que son utilisation soit plus simple pour pouvoir ajouter la possibilité d'y accéder depuis le menu principal du jeu et permettre à l'utilisateur d'inventer des niveaux.
- Une nouvelle modélisation de l'eau qui s'écoule moins aléatoirement.
- Visualisation de la zone d'explosion pour la dynamite.
- Création de nouveaux matériaux.
- Mise en place de vent.